

Geração de código Intermediário

MEPA: parte 1

Fabio Lubacheski
fabio.lubacheski@mackenzie.br

Geração de código intermediário

- Em razão da complexidade da geração de código, um compilador quebra essa fase em vários passos e frequentemente requer **estruturas de dados intermediárias** e alguma forma de **código abstrato (código intermediário)**.
- O código intermediário é útil quando o objetivo do compilador é produzir código eficiente (otimizado), além disso o código intermediário facilita a geração de código alvo (linguagem de máquina), tornando o compilador independente da máquina alvo.
- Temos duas representações frequentes de código intermediário: **código de três endereços** e **P-código**.

Código de três endereços

- A instrução básica do código de três endereços é projetada para representar a avaliação de expressões aritméticas com seguinte forma geral:
$$x = y \text{ op } z$$
onde **op** é um operador aritmético, exemplo + ou -, e **x** é obrigatoriamente um endereço de memória e **y** e **z** podem representar endereços de memória, constantes ou literais sem endereços na memória.
- O nome “código de 3 endereços” vêm do formato da instrução, na qual ocorre a manipulação de até 3 endereços na memória: **x**, **y** e **z**.

P-código

- O **P-código** usado inicialmente como código intermediário para compiladores da linguagem Pascal nos anos 70, foi projetado como um código para uma máquina hipotética denominada **P-máquina** baseada na **estrutura de dados pilha**, ou seja, não tínhamos registradores reais.
- Para executar o código foram implementados interpretadores que executavam o **P-código** diretamente em máquinas reais.
- Trazendo para os dias atuais podemos dizer que o **bytecode** da linguagem Java é uma linguagem intermediária **P-código** e a **JVM** é uma **P-máquina** que executa o código **bytecode** em máquinas reais.

<https://www.alura.com.br/artigos/jvm-conhecendo-processo-execucao-de-codigo>

<https://www.guiadojava.com.br/2022/03/uma-questao-de-interpretacao-de.html>

Como é o bytecode na linguagem Java

Quando compilamos um programa “Nome classe.java” é gerado um arquivo “Nome classe.class” que não legível, pois o **bytecode** é o formato que a **JVM** entende.

Para gerar os mnemônicos de um programa binário .class basta utilizar a o programa javap que acompanha o JDK.

Exemplo:

```
javap -c “Nome classe.class”
```

```
class Main {  
    public static void main(String[] args) {  
        System.out.println("Ola mundo.");  
    }  
}
```

```
1  Compiled from "Main.java"  
2  class Main {  
3      Main();  
4      Code:  
5          0: aload_0  
6          1: invokespecial #1  
7          4: return  
8  
9      public static void main(java.lang.String[]);  
10     Code:  
11         0: getstatic      #2  
12         3: ldc           #3  
13         5: invokevirtual #4  
14         8: return  
15 }
```

P-Máquina para Pascal

Para geração de código intermediário utilizaremos um computador hipotético (**P-máquina**), proposto pelo professor **Tomasz Kowaltowski** no livro Implementação de Linguagem de Programação que gera código para linguagem **Pascal**:

<https://sites.google.com/unicamp.br/ic-mo403-mc900>

MEPA – **M**áquina de **E**xecução para **P**ascal

- Características:
 - Máquina a pilha
 - Memória composta por 2 regiões:
 - ❖ Região do programa **P[]** (**programa**) que armazena as instruções da MEPA
 - ❖ Região da pilha ou dados **M[]** (**memória**) que armazena os valores manipulados pelas instruções da MEPA.

P-código da MEPA

A MEPA possui dois registradores especiais que serão utilizados para descrever os efeitos das instruções:

- ❑ O registrador de programa i contém o endereço da próxima instrução a ser executada em P , ou seja, $P[i]$.
- ❑ O registrador s indica o elemento no topo da pilha de memória, ou seja, $M[s]$.

Exemplo de instruções, P-código da MEPA:

CRCT **k** (carregar constante)

$s \leftarrow s + 1$

$M[s] \leftarrow \mathbf{k}$

$i \leftarrow i + 1$

CRVL **n** (carregar valor
de variável)

$s \leftarrow s + 1$

$M[s] \leftarrow M[\mathbf{n}]$

$i \leftarrow i + 1$

Instruções aritméticas

SOMA (soma)

```
M[s-1] <- M[s-1] + M[s]  
s <- s - 1  
i <- i + 1
```

SUBT (subtração)

```
M[s-1] <- M[s-1] - M[s]  
s <- s - 1  
i <- i + 1
```

MULT (multiplicação)

```
M[s-1] <- M[s-1] * M[s]  
s <- s - 1  
i <- i + 1
```

DIVI (divisão)

```
M[s-1] <- M[s-1] / M[s]  
s <- s - 1  
i <- i + 1
```

INVR (inverte valor)

```
M[s] <- -M[s]  
i <- i + 1
```

ARMZ **n** (armazenar valor)

```
M[n] <- M[s]  
s <- s - 1  
i <- i + 1
```

Como fica e executa uma expressão aritmética em MEPA ?

- Considere a expressão abaixo na notação infixa (operando operador operando) em uma linguagem alto nível:

$c := a + b$

- Considere que o **compilador alocou** as variáveis a , b , c estão nos seguintes endereços na memória **M**

$a = 00$ $b = 02$ $c = 01$

- Os valores armazenados nos endereços acima seriam

$a = 10$ $b = 20$ $c = ?$

- Qual seria o estado da memória **M[]** após a execução do trecho acima? Considere que o registrador **s** começa na posição **02** na memória **M[]**.

Passos para converter Infixa para Posfixa

1º passo: converte a expressão da notação infix a para posfixa

Para traduzir uma expressão na notação infix a podemos usar os sites abaixo:

Algoritmo para converter infix a para posfixa

<http://www.cs.man.ac.uk/~pjj/cs212/fix.html>

Site para converter infix a para posfixa

<https://raj457036.github.io/Simple-Tools/prefixAndPostfixConverter.html>

Site para gerar a árvore:

<https://Inogueir.github.io/expression-tree-gen/>

2º passo: traduz a expressão na notação posfixa para as instruções da MEPA na memória $P[]$.

3º passo: faz a execução das instruções armazenadas na memória $P[]$ utilizando $M[]$ como memória auxiliar.

Exercício

1) Considere a expressão abaixo na notação infixa (operando operador operando):

$$a := a + (b / 9 - 3) * c$$

Considere que as variáveis a , b , c estão nos seguintes endereços na memória M

$$a = 00 \quad b = 02 \quad c = 01$$

Os valores armazenados nos endereços acima seria

$$a = 10 \quad b = 18 \quad c = 2$$

Qual seria o estado da memória $M[]$ após a execução do trecho acima?

Considere que o registrador s começa na posição 02 na memória $M[]$.

Exercício

2) Considere as seguintes variáveis, com os seguintes endereços de memória e o registrador *s* na posição 03 na memória *M[]*.

a=03 *b*=01 *c*=00 *d*=02

com os seguintes valores armazenados

a=10 *b*=3 *c*=12 *d*=2

2.1) Faça a tradução das seguintes expressões para instruções da MEPA

a) $d + (b + 3) * (c - a) * 2$

b) $((b * d / 3) - 4 * b) + 8 / d$

2.2) Faça a execução das instruções apresentando o estado da memória *M[]* após a execução das expressões acima.

Análise sintática e Geração de Código - EAD

A principal vantagem do Analisador Sintático Descendente Recursivo (ASDR) é a simplicidade de sua implementação e sua flexibilidade.

Para que o nosso compilador faça a geração de código de programa-fonte para MEPA utilizaremos o ASDR já desenvolvido, e a partir dele, introduziremos rotinas que geram instruções em MEPA.

Imagine o problema de gerar código para expressões aritméticas com operandos com variáveis e números com somente um dígito e os operadores aritméticos (“+”, “-”, “*”, “/”). A gramática deve atender as prioridades dos operados, ou seja, em uma expressão com soma “+” e multiplicação “*”, a multiplicação deve ser resolvida primeiro (Ex: $1+a*3$). Além disso é possível o uso de parênteses para modificação de prioridades de operadores em expressão (Ex: $(1+a)*3$).

Análise sintática e Geração de Código – Definição da gramática

Considere a gramática abaixo na notação EBNF com símbolo inicial <expressão>, observe que nessa gramática os <não-terminais> são nomes entre parênteses angulares < e > e os símbolos **terminais** estão entre apóstrofo ". A gramática consegue reconhecer expressões aritméticas conforme a especificação anterior.

$$\langle \text{expressão} \rangle ::= \langle \text{termo} \rangle \{ ('+' | '-') \langle \text{termo} \rangle \}$$
$$\langle \text{termo} \rangle ::= \langle \text{fator} \rangle \{ ('*' | '/') \langle \text{fator} \rangle \}$$
$$\langle \text{fator} \rangle ::= 'a' | 'b' | 'c' | \dots | '1' | '2' | '3' | \dots | '(\text{ expressão })'$$

Análise sintática e Geração de Código - ASDR

Na implementação do ASDR teríamos as seguintes funções:

```
void expressao(){
    termo();
    while( lookahead == '+' ||
           lookahead == '-'){
        consome(lookahead);
        termo();
    }
}

void termo(){
    fator();
    while( lookahead == '/' ||
           lookahead == '*'){
        consome(lookahead);
        fator();
    }
}
```

```
void fator(){
    if(isalpha(lookahead) ||
       isdigit(lookahead))
        consome(lookahead);
    else{
        consome('(');
        expresao();
        consome(')');
    }
}
```

Análise sintática e Geração de Código – ASDR + posfixa

Imagine agora que queremos que o Analisador Sintático tenha como saída a expressão aritmética convertida para **notação posfixa**. Isto pode ser conseguido facilmente inserindo comandos apropriados em alguns pontos nas funções mutuamente recursivas.

```
void expresao(){
    termo();
    while( lookahead == '+' ||
           lookahead == '-' ){
        char op = lookahead;
        consome(lookahead);
        termo();
        printf("%c",op);
    }
}
```

```
void termo(){
    fator();
    while( lookahead == '*' ||
           lookahead == '/' ){
        char op = lookahead;
        consome(lookahead);
        fator();
        printf("%c",op);
    }
}
```

Análise sintática e Geração de Código – ASDR + posfixa

```
void fator(){  
    if(isalpha(lookahead) ||  
        isdigit(lookahead)){  
        printf("%c", lookahead);  
        consome(lookahead);  
    }  
    else{  
        consome(' ( ');  
        expressao();  
        consome(' ) ');  
    }  
}
```

Análise sintática e Geração de Código – ASDR + MEPA

Para que o Analisador Sintático passe a gerar código MEPA para a gramática anterior, basta trocar a impressão dos terminais da gramática por instruções da MEPA equivalente.

```
void expressao(){
    termo();
    while( lookahead == '+' ||
           lookahead == '-' ){
        char op = lookahead;
        consome(lookahead);
        termo();
        if( op == '+' )
            printf("SOMA\n");
        else
            printf("SUBT\n");
    }
}
```

```
void termo(){
    fator();
    while( lookahead == '*' ||
           lookahead == '/' ){
        char op = lookahead;
        consome(lookahead);
        fator();
        if( op == '*' )
            printf("MULT\n");
        else
            printf("DIVI\n");
    }
}
```

Análise sintática e Geração de Código – ASDR + MEPA

```
void fator(){  
    if(isalpha(lookahead)){  
        printf("CRVL %c\n",lookahead);  
        consome(lookahead);  
    }  
    else if(isdigit(lookahead)){  
        printf("CRCT %c\n",lookahead);  
        consome(lookahead);  
    }else{  
        consome('(');  
        expressao();  
        consome(')');  
    }  
}
```

Muito importante para aprender mais !!!

Leia o capítulo 8 (pag 93) do livro **Implementação de Linguagens de Programação** do professor Tomasz Kowaltowski disponível em:

<https://drive.google.com/file/d/1DyqeBUgayQpjh1yaHziQgm8WCE7u7ZaN/view>

Grato !

Bons estudos !